

Progetto di Calcolo Parallelo 2025/2026

Nicolas Cola, matr. 0001235951

18/09/2026

Introduzione

Questo documento è volto a presentare i dettagli e le scelte implementative riguardanti la parallelizzazione del programma per il calcolo del correlogramma.

Sono presentate due versioni del programma:

- La prima sfrutta il parallelismo a memoria condivisa tramite l'utilizzo di OpenMP
- La seconda sfrutta il parallelismo a memoria distribuita grazie a MPI

Entrambe le versioni sono state testate sul cluster di dipartimento sulla partizione rtx2080. Il cluster impone limitazioni sulle risorse computazionali utilizzabili, quindi i risultati presentati saranno vincolati da esse. In particolare, tali limitazioni riguardano:

- Max 2 nodi per utente
- Max 4 CPU per nodo

Versione OpenMP

Utilizzando OpenMP il programma è stato parallelizzato in tre diversi punti: nel calcolo della media, della varianza e del coefficiente di correlazione.

Per quanto riguarda la media e la varianza, entrambe sono riduzioni $O(n)$ su X . Pertanto, la loro parallelizzazione è stata realizzata con l'utilizzo della direttiva `#pragma omp parallel for` utilizzando le clausole `reduction(+:mean)` e `reduction(+:var)` rispettivamente per media e varianza. Così facendo, ogni thread accumula una somma parziale privata, che viene poi combinata nella variabile condivisa grazie alla barriera implicita al termine del ciclo.

La parallelizzazione del calcolo del coefficiente di correlazione ha richiesto un'analisi più attenta. In particolare, 3 differenti strategie di parallelizzazione sono state prese in considerazione:

1. **Parallelizzazione del ciclo interno su nvalues.** Ciò sarebbe possibile utilizzando la clausola `reduction(+:ac)`, ma occorre porre attenzione a come si crea il pool di thread da distribuire sul ciclo. Se si utilizzasse la direttiva `#pragma omp parallel for` direttamente sul ciclo, la parallelizzazione risulterebbe sconsigliata dato che il team di thread verrebbe ricreato e distrutto ad ogni iterazione del ciclo esterno, introducendo un notevole overhead.

Al contrario, utilizzando la direttiva `#pragma omp parallel` prima del ciclo esterno e distribuendolo ad ogni iterazione di quest'ultimo tramite `#pragma omp for reduction(+:ac)`, l'overhead dovuto alla creazione del team di thread sparirebbe e la parallelizzazione risulterebbe conveniente.

Tuttavia, la dichiarazione della variabile `ac` sarebbe all'interno della regione parallela, quindi privata a ciascun thread, ma, per poter applicare la reduction, dovrebbe essere condivisa tra tutti i thread. Pertanto, per poter applicare quest'ultimo approccio, sarebbe necessario

spostare la dichiarazione di `ac` fuori dalla regione parallela in modo da renderla condivisa tra i thread, e inserire direttive di sincronizzazione sulla sua inizializzazione e sulla scrittura dell'array `coef`. Tutto ciò introdurrebbe un notevole overhead che rende anche questo approccio inefficiente.

2. **Parallelizzazione del ciclo esterno su `maxshifts`.** Fattibile utilizzando la direttiva `#pragma omp parallel for` sopra il ciclo esterno, ogni thread eseguirebbe in modo indipendente dagli altri il ciclo interno. Non ci sono race conditions dato che la variabile `ac` è dichiarata localmente nel ciclo, quindi anche privata ad ogni thread. Inoltre, anche la scrittura del valore finale nell'array `coef` non presenta race condition in quanto ogni thread scrive in posizioni di memoria differenti.
3. **Parallelizzazione di entrambi i cicli.** Con l'utilizzo della clausola `collapse(2)` nella direttiva `#pragma omp parallel for` sul ciclo esterno, entrambi i cicli verrebbero collassati in uno solo e parallelizzato tra i vari thread. Tuttavia, questo approccio risulta praticabile solo se:
 - L'inizializzazione della variabile `ac` tra i due cicli venga spostata prima del ciclo esterno e venga azzerata dopo ogni scrittura nell'array `coef`
 - Si introducano costrutti di sincronizzazione per evitare race conditions sulla scrittura dell'array `coef`. Questo introdurrebbe un notevole overhead che renderebbe tale approccio di parallelizzazione molto sconsigliato e inefficiente.

Le considerazioni emerse dall'analisi effettuata e i risultati empirici ottenuti testando i tempi di esecuzione sul cluster di dipartimento fanno sì che la strategia scelta per la parallelizzazione del calcolo del coefficiente di correlazione sia la 2. Infatti, i tempi di esecuzione maggiori delle strategie 1 e 3 sono dovuti alla presenza di direttive di sincronizzazione in esse, rendendo la strategia 2 migliore.

Versione MPI

Nella versione del programma a memoria distribuita con MPI, si è scelto di adottare un parallelismo a livello di task rispetto ad un parallelismo a livello di dato.

Solo il processo di rango 0 chiama `read_signal` per inizializzare l'array `X` e la variabile `nvalues`. Quest'ultima viene diffusa tramite `MPI_Bcast` a tutti gli altri processi, consentendo loro di allocare `X` con la dimensione corretta prima che venga diffuso l'array stesso. Così facendo, ogni processo possiede una copia identica del segnale, senza ulteriori comunicazioni riguardanti `X`.

Per quanto riguarda il calcolo della media e della varianza, questo viene eseguito in modo seriale da tutti i processi in parallelo. Inizialmente è stata testata una versione basata sulla suddivisione del carico tramite `MPI_Allreduce`, dove ogni nodo somma una porzione locale di `X` per poi combinare le somme parziali. Tuttavia, testando le due versioni sul cluster di dipartimento, quest'ultima versione ha mostrato tempi di esecuzione maggiori, dovuti all'overhead di comunicazione tra i nodi che annullano il vantaggio derivante dalla parallelizzazione dei calcoli.

Per la parallelizzazione del calcolo del coefficiente di correlazione, si è scelto di parallelizzare il ciclo esterno piuttosto che il ciclo interno.

Per garantire un bilanciamento del carico tra i nodi, il ciclo esterno viene suddiviso utilizzando la formula vista a lezione. Questo garantisce che la suddivisione del lavoro sia bilanciata, senza la necessità che `maxshifts` sia un multiplo del numero di nodi (`comm_size`).

Ogni nodo calcola una partizione locale dell'array `coef` in array `local_coef` di lunghezze potenzialmente diverse, risultati parziali vengono raccolti dal nodo con rango 0 utilizzando `MPI_Gatherv`

nell'array globale `coef`. A differenza di `MPI_Gather`, l'utilizzo di `MPI_Gatherv` permette la ricezione di messaggi di dimensione variabile, adattandosi alla logica di scomposizione a blocchi adottata.

Le motivazioni per la scelta della parallelizzazione del ciclo esterno sono dovute principalmente all'overhead di comunicazione che si introdurrebbe parallelizzando il ciclo interno, che degraderebbe notevolmente le prestazioni. Ciò è causato da:

1. Necessità di eseguire una `MPI_Allreduce` al termine del calcolo di ogni singolo lag h per sommare i risultati parziali, richiedendo migliaia di comunicazioni. Parallelizzando invece il ciclo esterno, l'algoritmo diventa imbarazzantemente parallelo e ogni nodo calcola indipendentemente un sottoinsieme di coefficienti senza la necessità di ulteriori comunicazioni.
2. L'accesso circolare agli indici tramite l'operatore modulo (`X[(i+h) % nvalues]`) richiesto dalla formula dell'autocorrelazione. Se l'array `X` fosse stato suddiviso tra i vari nodi, all'aumentare di h i processi avrebbero dovuto richiedere continuamente dati appartenenti alla memoria di altri processi, rendendo necessario uno scambio di ghost-cells. Facendo in modo che tutti i nodi abbiano una copia di `X` risolve questo problema. Avendo i dati in memoria locale, il calcolo della partizione `local_coef` avviene senza alcuno scambio di messaggi.

Analisi teorica del programma OpenMP

Si parte prima da un'analisi del costo computazionale dell'algoritmo seriale come base, a partire da questa si analizzano Work e Span del programma parallelizzato tramite OpenMP.

Siano $n = \text{nvalues}$ e $m = \text{maxshifts}$. Il programma seriale può essere suddiviso logicamente in tre parti:

1. Calcolo della media = $O(n)$
2. Calcolo della varianza = $O(n)$
3. Calcolo del coefficiente di correlazione = $O(m \cdot n)$

Il costo computazionale totale dell'algoritmo seriale è $2 \cdot O(n) + O(m \cdot n) = O(m \cdot n)$.

Per analizzare Work e Span dell'algoritmo parallelo occorre prima di tutto fare alcune assunzioni:

- p processori
- $p \ll n$ e $p \ll m$
- una riduzione richiede Work = p e Span = $\log p$ alla fine del ciclo for

Anche per l'analisi di Work e Span suddividiamo l'algoritmo in tre parti da analizzare separatamente:

1. Calcolo della media:

- *Work*: ogni processore esegue n/p somme seguite da una combinazione ad albero delle somme parziali. Pertanto, il lavoro è $T_1(n) = p \cdot n/p + p = n + p$. Essendo $p \ll n$, p diventa trascurabile e $T_1(n) = n$.
- *Span*: un singolo processore esegue n/p somme e una $\log p$ combinazione. Di conseguenza lo Span è $T_p(n) = n/p + \log p = (n + p \log p)/p$. Siccome $p \log p = O(n)$, $T_p(n) = n/p$.

2. Calcolo della varianza: come per la media

3. Calcolo del coefficiente di correlazione:

- *Work*: ogni processore esegue m/p iterazioni del ciclo esterno e n del ciclo interno. Il lavoro è $T_1(m, n) = p \cdot (m/p) \cdot n = m \cdot n$.
- *Span*: $T_p(n, m) = (m/p) \cdot n$

Pertanto, il Work del programma parallelo è $T_1(n) = 2 \cdot n + m \cdot n = m \cdot n$, mentre lo Span totale è $T_p(n, m) = 2 \cdot (n/p) + (m/p) \cdot n = (m/p) \cdot n$.

In fine, lo speed-up e l'efficienza derivati da Work e Span sono calcolati come:

- $S(p, n, m) = \text{Work}/\text{Span} = (n \cdot m)/((m/p) \cdot n) = p$
- $E(p, n, m) = \text{Speed-up}/p = p/p = 1$

Ciò significa che, fino a quando le assunzioni fatte all'inizio rimangono valide, otteniamo uno speed-up lineare e l'efficienza massima. Le metriche ottenute sono pertanto ideali, indicando un ottimo uso delle risorse computazionali e un'ottima scalabilità del programma parallelo al variare del numero di processori.

Analisi empirica dei programmi paralleli

In questa sezione verranno discussi i risultati ottenuti dall'analisi empirica delle due versioni del programma parallelizzato tramite OpenMP e MPI.

Entrambe le analisi sono state effettuate sul cluster di dipartimento misurando i tempi di esecuzione al variare del numero di processori e della dimensione dell'input.

Per ottenere risultati più precisi, le metriche vengono calcolate utilizzando la media di 20 tempi di esecuzione del programma.

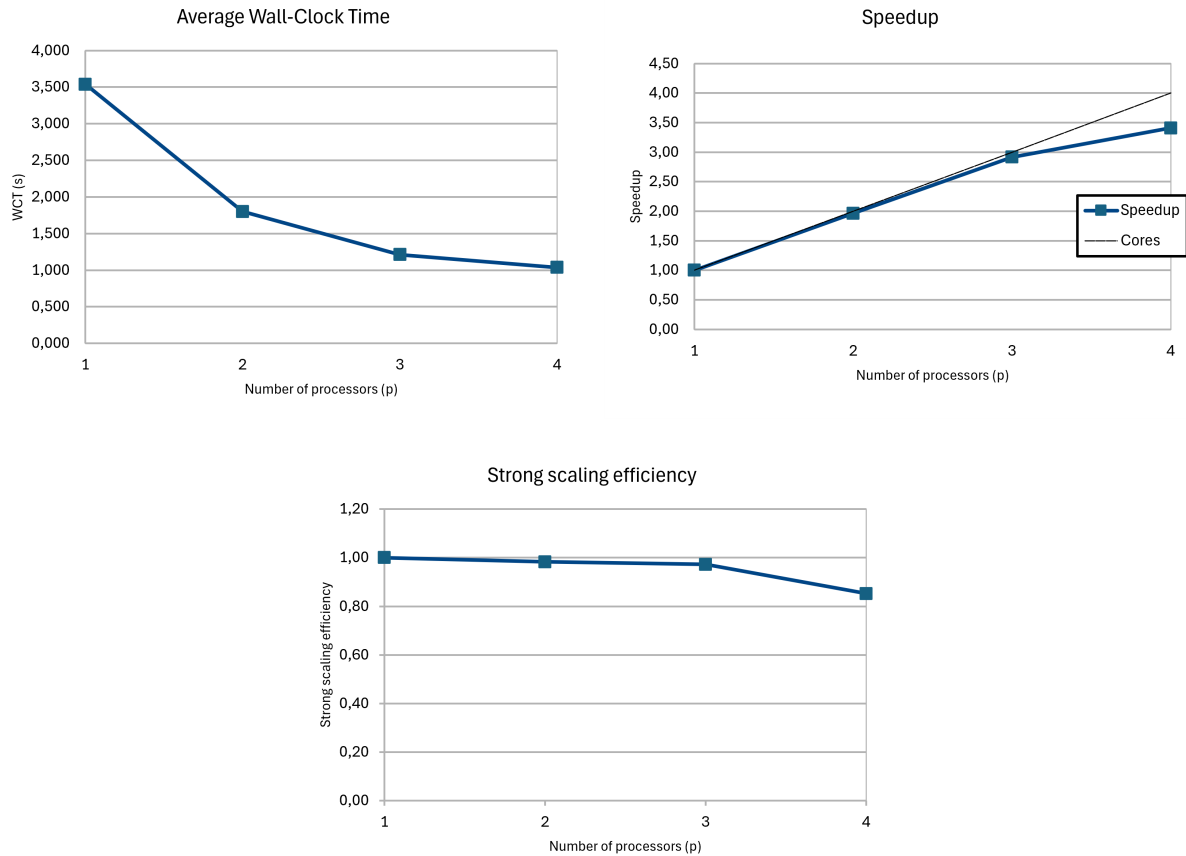
In particolare, per ognuna delle due versioni, per prima cosa è stato analizzato come il tempo medio di esecuzione, lo speed-up e la strong-scaling efficiency variassero al variare del numero di processori, mantenendo la dimensione degli input fissa a `nvalues` = 10^6 e `maxshifts` = 2048.

Successivamente, è stata misurata la weak-scaling efficiency variando il valore di `maxshifts` in modo proporzionale al numero di processori utilizzati (`maxshifts` = $2048 \cdot p$), mantenendo però la dimensione di `nvalues` fissa a 10^6 . Per la versione MPI, la ragione per cui viene aumentato solamente `maxshifts` è dovuta al fatto che la computazione è parallelizzata solamente nel ciclo esterno su `maxshifts` del calcolo del coefficiente di correlazione, lasciando il ciclo interno e i due cicli per il calcolo di media e varianza su `nvalues` strettamente seriali. Invece, nella versione OpenMP, in cui anche i cicli di media e varianza vengono parallelizzati, si è scelto di scalare solamente `maxshifts` in quanto, a seguito dell'analisi teorica del programma, è emerso che la computazione dominante è quella del calcolo del coefficiente di correlazione. Ciò implica che il lavoro per processore effettuato nel calcolo di media e varianza ($2 \cdot (n/p)$) diventa trascurabile rispetto a quello del calcolo del coefficiente di correlazione ($(m/p) \cdot n$), facendo sì che variando solamente `maxshifts` il lavoro per processore rimanga costante al variare del numero di processori.

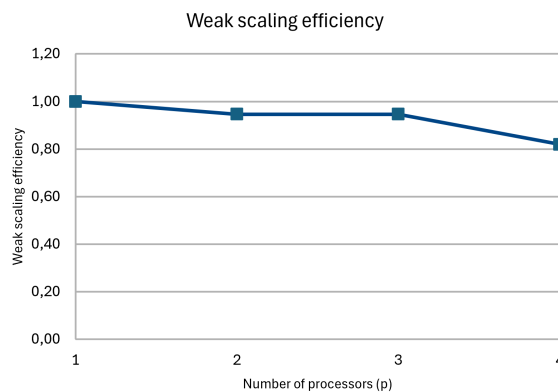
Versione OpenMP

Per quanto riguarda la versione OpenMP, il programma è stato eseguito su un singolo nodo, con un singolo task e con un range da 1 a 4 processori.

Come si può vedere dai seguenti grafici, il programma scala in modo quasi lineare fino a 3 processori e raggiunge il tempo di esecuzione più breve e lo speedup massimo con 4 processori. Tuttavia, l'utilizzo del quarto processore peggiora l'efficienza. Ciò indica un effettivo peggioramento nell'utilizzo delle risorse computazionali, probabilmente dovuto al fatto che, poichè vengono utilizzati tutti i processori disponibili, il sistema operativo è costretto a bloccare i threads per eseguire altri processi di sistema.



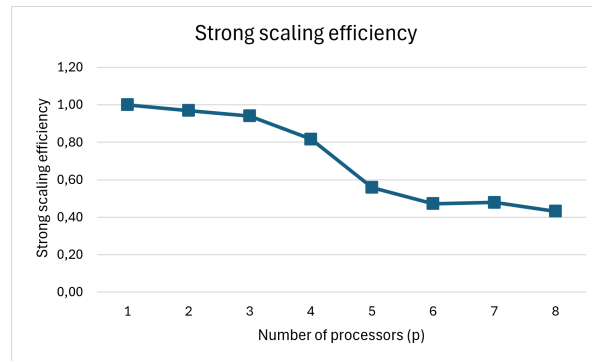
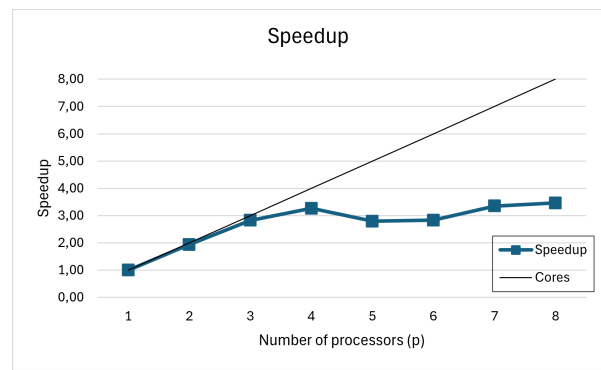
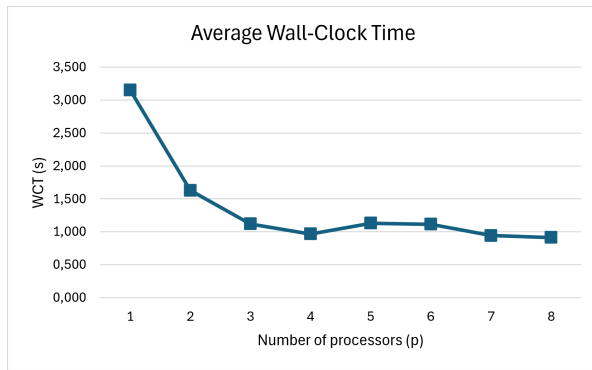
Per quanto riguarda la weak-scaling efficiency, come suggerisce il seguente grafico, il programma scala perfettamente variando la dimensione di `maxshifts` proporzionalmente al numero di processori. Ciò dimostra come il programma possa essere eseguito nello stesso tempo nonostante la maggiore dimensione dell'input. L'unico degrado nelle prestazioni che si nota dal grafico introducendo il quarto processore può essere attribuito allo stesso motivo del degrado notato nella strong-scaling efficiency.



Versione MPI

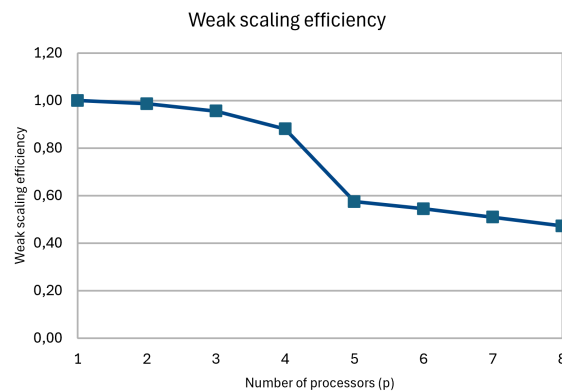
La versione parallela a memoria distribuita con MPI viene eseguita utilizzando da 1 a 8 processori. Poichè il numero di processori è limitato a 4 per nodo, quando se ne utilizzano più di 4, il programma viene eseguito su 2 nodi.

I risultati ottenuti sono presentati nei seguenti grafici:



Fino a 4 processori, le metriche sono simili a quelle della versione OpenMP, indicando una scalabilità quasi lineare fino a 3 processori. Con l'introduzione del quinto processore, e quindi del secondo nodo, il tempo d'esecuzione aumenta, lo speedup diminuisce e l'efficienza cala drasticamente. Ciò è causato dall'overhead di comunicazione tra i due nodi che devono comunicare per sincronizzarsi e scambiare dati attraverso la rete che li interconnette. Dal sesto all'ottavo nodo, le metriche si stabilizzano e migliorano lievemente, raggiungendo il tempo di esecuzione minimo e lo speedup massimo con 8 processori. Tuttavia, con l'introduzione dell'ultimo processore, anche l'efficienza raggiunge il valore minimo. Questo è probabilmente dovuto allo stesso motivo della versione OpenMP. In entrambi i nodi, in cui tutti i processori sono impiegati per l'esecuzione del programma, il sistema operativo interrompe i thread per eseguire processi di sistema.

Infine, come si può vedere dal seguente grafico rappresentante la weak-scaling efficiency, essa ha un andamento simile alla strong-scaling efficiency. Ciò suggerisce che al variare della dimensione dell'input e del numero di processori impiegati, il programma scala bene ma subisce un degrado delle prestazioni causato dalle motivazioni precedentemente discusse.



Conclusioni

Confrontando i risultati ottenuti dalle analisi empiriche delle due versioni del programma parallelo, a parità di processori impiegati ($p = 4$), le prestazioni sono molto simili e non mostrano grandi differenze. Introducendo gli ulteriori processori per la versione MPI, nonostante ciò dimostri un grande calo in termini di efficienza, il programma raggiunge lo speedup massimo e il tempo di esecuzione minimo in termini globali per i due programmi. Tuttavia, quest'ultima comparazione non dimostra una superiorità della versione MPI rispetto a quella OpenMP poichè la quantità di risorse computazionali non è la stessa.